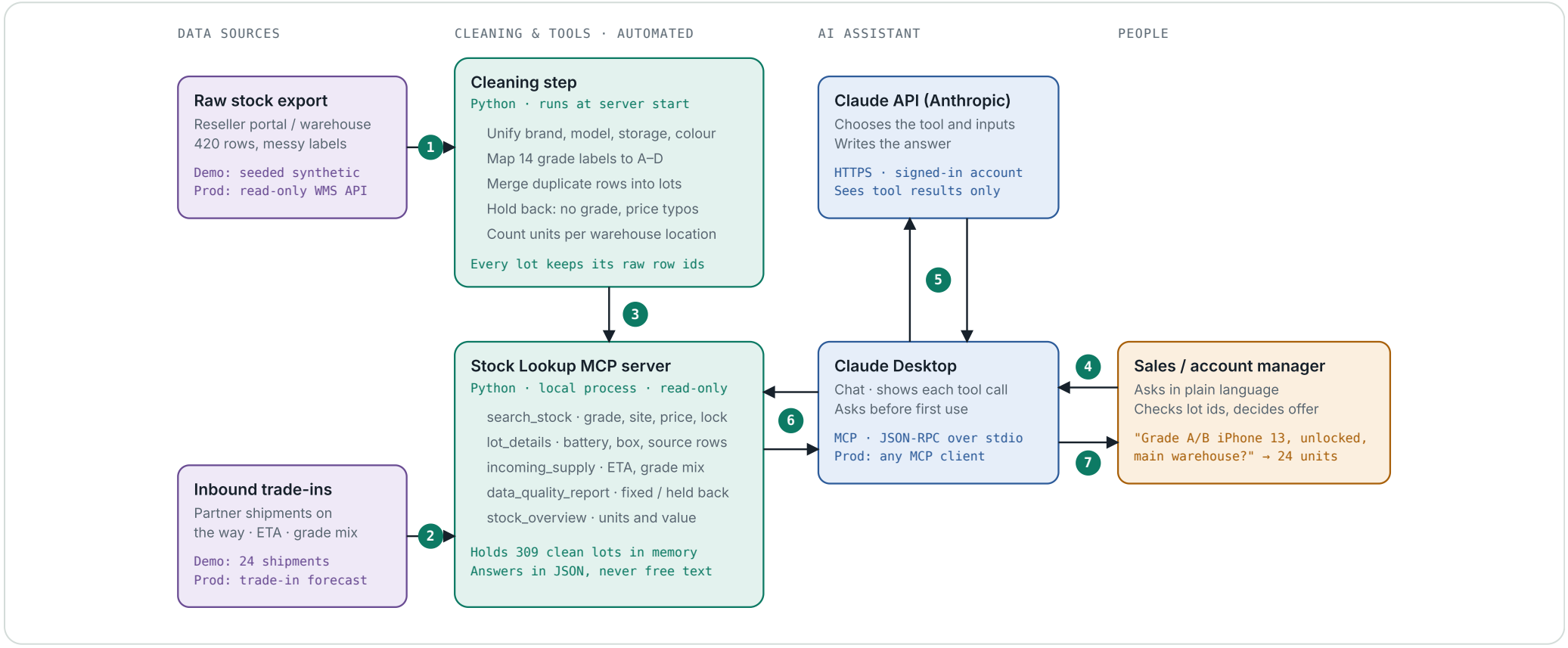
PHONE RECOMMERCE · AI USE CASE PROTOTYPE

Stock Lookup: how a sales question becomes a checked answer from clean stock data

Raw stock exports list the same phone many times, with mixed grade labels, colour names and price typos, so buyers and sales cannot filter them reliably. This prototype cleans the export into sellable lots and gives Claude five read-only lookup tools through the Model Context Protocol (MCP). A salesperson asks in plain language; Claude answers from the tools, with lot numbers that trace back to the raw rows.

INFORMATION FLOW

From raw export to a traceable answer in seven steps



Automated logic (built) · External platform · Person · Data source

STEP BY STEP

What moves where, and over which connection

STEP	FROM → TO	WHAT MOVES	PLATFORM & CONNECTION	PROTOTYPE TODAY	IN PRODUCTION (PROPOSED)
1	Stock export → Cleaning	Device rows: brand, model, grade label, colour, lock, battery, quantity, price, location	`Read-only file / API`	420 seeded synthetic rows with realistic errors	Nightly or hourly read from the warehouse / ERP system with a read-only service account
2	Trade-ins → Server	Inbound shipments: partner, model, units, ETA, expected grade mix	`Read-only file / API`	24 synthetic shipments over 30 days	Trade-in platform and supply forecast
3	Cleaning → Server	Clean lots with units per location, held-back rows with reasons, quality report	`In memory`	Rebuilt each time the server starts	Scheduled job writes a clean table the server reads
4	Person → Claude Desktop	The question in plain language	`Chat`	Claude Desktop on the laptop	Claude for the sales team, or a buyer assistant on the portal
5	Claude Desktop ↔ Claude API	Question and tool list in; chosen tool and inputs out; later the tool result in and the answer out	`Anthropic API` `HTTPS`	Personal Claude account	Company Claude plan under the AI policy
6	Claude Desktop ↔ MCP server	`tools/call search_stock` with filters; JSON result with lots, units at the site, lot ids	`MCP` `JSON-RPC · stdio`	Local Python process started by Claude Desktop	Hosted MCP server over HTTPS with company sign-in
7	Claude Desktop → Person	Answer with lot ids; follow-ups with `lot_details` or `incoming_supply`	`Chat`	Shown with the tool call visible	Same, plus a link to the lot in the stock system

CONTROLS

Why it is safe to pilot

Read-only by design

The five tools only look things up. No tool can change stock, prices, reservations or orders.

Cleaned before Claude sees it

Claude never reads raw rows, so it cannot repeat a price typo or offer an ungraded device as grade A.

Every answer is traceable

Each lot lists the raw rows it came from. `lot_details` shows them, so sales can check any number.

Numbers come from code

Counts, totals and grade splits are calculated by the server. Claude only words the answer.

Tested before trusted

A self-test checks every tool and that site totals add up. It caught a double count in the first version (39 instead of 24).

Little data leaves the building

Only the tool result for the current question goes to the Claude API. No customer personal data is involved; the demo data is synthetic.

No secrets, no open ports

The server is a local process with no API keys and no network listener. Production would use a read-only service account.

People decide the offer

Claude answers stock questions. The salesperson decides what to quote and reserve.

Prototype by Abel Louis on synthetic data (fictional partners, devices and prices). Code: github.com/abellouisr/recommerce-stock-lookup-mcp